

2025 年中国研究生数学建模竞赛

A 题

通用神经网络处理器下的核内调度

基于驻留段约束的缓存与流水协同优化

2025 年 A 题历史真题研究

基于驻留段约束的缓存与流水协同优化

摘 要

针对神经网络处理器计算图中拓扑依赖、有限连续缓存和多执行单元并行之间的耦合，建立由拓扑序优化、驻留段分配和流水重排构成的调度方法。以题目提供的六个计算图为对象，分别评价最大缓存驻留量、额外数据搬运量和总执行周期。

对于最小驻留问题，将缓存申请延迟至首次使用之前，并在满足全部依赖后尽早释放，再比较编号顺序、有限就绪窗口贪心和前驱深度优先顺序。卷积小图的驻留峰值由 7560 降至 6984，其余五图保留生命周期收缩后的编号基准。实验表明，仅依据当前新增与释放量的局部贪心可能导致更差的全局峰值，因而采用多候选筛选而非固定使用单一规则。

对于物理缓存分配问题，以缓冲区驻留段为单位实施连续区间最佳适配，空间不足时采用下一次使用距离及搬运代价辅助选择换出对象。对受保护缓冲区造成的碎片，通过显式换出和重新装入恢复连续空间，全部代价计入评价。通过逐操作、逐地址的重放，验证容量、生命周期、依赖及使用时驻留条件。

对于性能优化问题，在额外搬运量不得增加的约束下，比较地址轮换策略，并将已有缓存方案转换为包含复用依赖的驻留图。固定驻留关系后，使用最早可开始时间与下游关键路径优先级重排执行单元指令。在额外搬运量保持不变时，六个计算图的总执行时间降低 1.30% 至 33.38%。注意力大图的执行时间由 225902 降至 150499 周期，额外搬运量均为 32852。该结果表明，在不增加缓存搬运开销的情况下，显式保留物理地址依赖并重新安排流水顺序能够改善执行效率。本文给出的结果为经约束验证的启发式可行解，不具有全局最优性保证。

关键词：有向无环图；缓存驻留；连续地址分配；换入换出；流水调度

1. 问题重述

题目将算子细化为由原子指令构成的有向无环图，节点包括指定执行单元上的操作，以及缓冲区申请和释放操作 [1]。每个缓冲区只属于一种缓存，需占据长度确定的连续地址区间。不同执行单元可以并行工作，同一执行单元只能串行执行。数据依赖和物理地址复用都会引入等待，因此拓扑顺序、缓存位置与实际完成时间必须联合考虑。

问题一要求在不考虑具体缓存容量限制时，寻找最大累计驻留量尽可能小的拓扑序。问题二在给定各类缓存容量后，为缓冲区确定偏移，必要时加入换出及换入操作，使额外搬运量尽可能小。问题三则允许进一步调整顺序和分配，在搬运量不显著增加的前提下降低执行时间。三个问题分别突出存储压力、物理可行性和执行并行性，目标之间存在竞争。

本文对六个输入图使用相同算法组合，不以算子名称编写专用调度规则。题面未给出“不显著增加”的数值阈值，因此问题三采用更严格的搬运量零增加约束。研究重点是获得可核验的完整调度方案，并明确启发式与评价模型的适用边界。

2. 数据理解与总体分析

2.1 输入结构与审计

输入同时提供节点和边的 JSON、CSV 表示。节点编号用于识别对象，不代表最优执行次序。操作节点的缓冲区列表同时包含输入与输出，不能仅依据列表顺序推断读写属性。本文直接使用题面给出的依赖边约束生产与消费先后。

六图规模如下。读取时逐字段比对两种表示，检查节点编号、重复边、缓存类型、大小及申请释放配对。全部输入通过检查，没有对缺失值进行填补，也没有删除任何节点或依赖。

表 1: 输入计算图规模

计算图	节点数	依赖边数
Conv_Case0	2580	3869
Conv_Case1	36086	85653
FlashAttention_Case0	1716	2712
FlashAttention_Case1	6952	11184
Matmul_Case0	4160	7104
Matmul_Case1	30976	55040

2.2 目标的耦合关系

缓存驻留峰值低，并不意味着每种缓存都能分配成功。一方面，不同类型不能互相借用空间；另一方面，分散的空闲区间无法容纳较大的连续申请。与此相反，过度追求小峰值可能使原本可重叠的计算串行化。因此首先通过生命周期收缩建立基准，再分别围绕搬运量和时间评价候选。

采用“拓扑顺序—连续分配—依赖重建—流水评价”的计算链。求解程序只产生候选，随后由不调用求解模块的重放程序从文本附件读回结果，重新检查空间和时间约束。两个程序共享输入字段定义，但不共享区间分配逻辑，减少同一实现错误同时影响生成和检查的可能性。

3. 模型假设与歧义处理

缓存大小沿用题面的抽象容量单位，不将其转换成字节；执行时间统一使用时钟周期。缓存申请与释放耗时为零，操作耗时固定为节点的周期字段。原始依赖边均不可删除，也不依据真实芯片的额外机制添加题面没有给出的吞吐假设。

问题一的驻留量按所有缓存类型合计计算，而非各类型峰值的最大值。对同一类型同时驻留的缓冲区要求地址不重叠，不同类型拥有独立地址空间。

附录 C 通过前一缓冲区的释放节点约束后一缓冲区的申请节点，直接描述了未发生换出的地址复用。发生换出时，物理地址在换出结束后即可复用，缓冲区的逻辑生命则持续到原始释放节点。本文将两者区分为逻辑生命周期与物理驻留段：申请或换入开启一段驻留，释放或换出结束该段。新驻留段必须等待所复用旧段的结束。若仍以最终释放节点约束换出期间的所有复用，可能人为形成依赖环，无法表达题面允许的换出机制。因此驻留段是本文对该情形的明确补充解释；与未提供的官方评测程序的一致性尚不能确证。

换出目标曾被任一输入复制节点引用时，按题面规定只计换入搬运量，换出时间为零；不自行判断数据是否被后续修改并改变计费方式。所有缓存类型均依题面统一换出规则处理。问题三要求额外搬运量不高于问题二选定方案，从而避免人为设定宽松的容忍阈值。

4. 符号说明

表 2: 主要符号

符号	含义
$G = (V, E)$	原始计算图及其依赖边
N, B	原始节点数、缓冲区集合
s_b, t_b, C_t	缓冲区大小、所属缓存类型及对应容量
a_b, f_b	缓冲区原始申请、释放节点
$\pi(v)$	节点在调度序列中的位置
x_{bj}	缓冲区第 j 个驻留段的起始偏移
H, D, T	最大驻留量、额外搬运量、总执行周期
c_v, p_v	节点执行周期、所属执行单元
S_v, F_v	节点实际开始、结束时间
r_v	从节点出发的下游最长耗时路径长度
W, α	就绪候选窗口、释放收益或新增量权重

5. 最小缓存驻留调度

5.1 目标与约束

对每个原始节点定义驻留增量 m_v ：申请节点取对应缓冲区大小，释放节点取其相反数，其余节点取零。任意有效拓扑序的累计驻留量为

$$H(\pi) = \max_{1 \leq k \leq N} \sum_{\pi(v) \leq k} m_v, \quad \pi(u) < \pi(v), \quad (u, v) \in E. \quad (1)$$

目标是最小化 $H(\pi)$ ，同时保证每个节点恰好出现一次。该表达式等价于对各缓冲区生命周期指示函数求和；不同缓存类型的峰值一般发生在不同位置，因此不能将分类峰值直接相加作为总峰值。

5.2 生命周期收缩与候选顺序

实际数据满足申请节点无入边、释放节点无出边的题面结构。固定操作节点顺序后，把申请推迟到第一次需要该缓冲区的位置之前，不会破坏已经完成的操作；把释放提前到最后一次使用且全部显式前驱完成之后，也不会影响后续操作。两种变换都不扩大任何生命周期，故累计驻留峰值不增加。程序对全部原始边再作检查，以覆盖申请到多个操作的显式依赖。

对操作诱导子图构建就绪集合。编号候选每次选择最小编号节点。缓存贪心从最小编号的 W 个就绪节点中，最小化

$$q(v) = n_v - \alpha d_v, \quad (2)$$

其中 n_v 是执行该节点需新申请的容量， d_v 是该节点执行后达到最后一次使用的缓冲区总大小。相同评分依次按新增容量、下游路径长度和编号排序。比较 $(W, \alpha) = (32, 1), (128, 1), (64, 2)$ 。该评分只评估局部收益，不对未完成的未来分支给出最优性承诺。

另从各终端操作出发，沿前驱进行深度优先遍历，按完成访问顺序形成拓扑序；前驱分别按编号正序、逆序访问形成两种候选。这一方法倾向于完成一个依赖分支后再进入下一个分支，但共享数据较多时也可能延长其他缓冲区生命。最终在全部候选中按实测 H 选择。

5.3 复杂度与结果

下游关键路径和深度优先遍历本身为 $O(N + |E|)$ 。当前缓存贪心实现每步扫描就绪集合获取窗口，保守时间上界为 $O(N^2 \log W + |E|)$ ，而非仅与窗口大小线性相关；图、缓冲区引用和运行状态的空间开销为 $O(N + |E| + \sum_v |\text{Bufs}_v|)$ 。每图节点最多约三万六千，采用有限候选而非全排列搜索。

表 3: 六图最大驻留量

计算图	编号基准	问题一结果
Conv_Case0	7560	6984
Conv_Case1	14048	14048
FlashAttention_Case0	4884	4884
FlashAttention_Case1	7972	7972
Matmul_Case0	9728	9728
Matmul_Case1	35328	35328

卷积小图从 7560 降至 6984；其余五图未找到比收缩后的编号基准更好的峰值，因而保留基准。图 1 反映的是本轮候选比较结果，不能据此认定其他图已达到理论下界。该现象也提示，输入编号可能已包含有利的局部组织，复杂启发式未必优于简单基准。

6. 连续缓存分配与换入换出

6.1 驻留段约束

缓存容量为 $C_{L1} = 4096$ 、 $C_{UB} = 1024$ 、 $C_{LOA} = C_{LOB} = 256$ 、 $C_{LOC} = 512$ 。对缓冲区 b 的每个驻留段 j ，要求

$$0 \leq x_{bj}, \quad x_{bj} + s_b \leq C_{tb}. \quad (3)$$

若两个同类型驻留段在调度序列中相交，则必须满足

$$x_{bj} + s_b \leq x_{ak} \quad \text{或} \quad x_{ak} + s_a \leq x_{bj}. \quad (4)$$

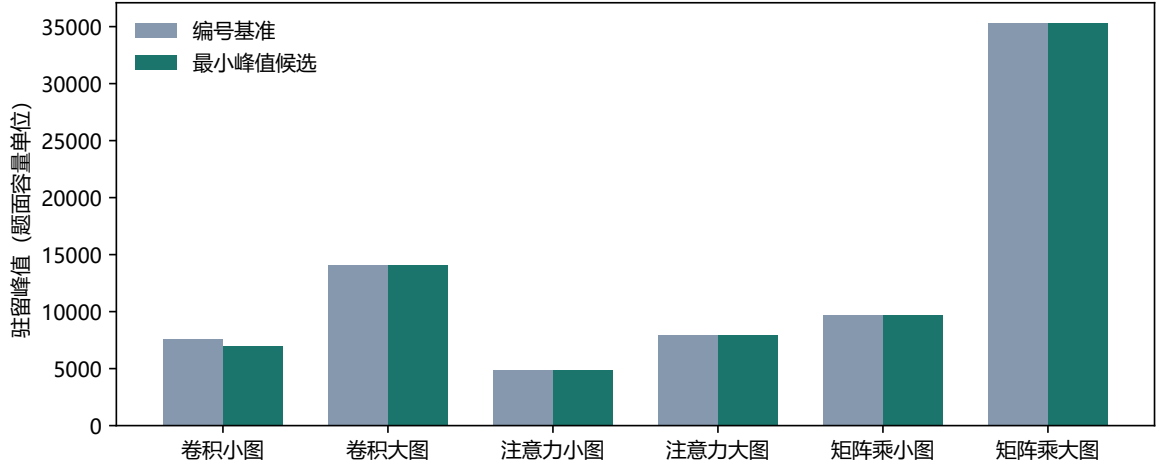


图 1: 生命周期收缩后的编号基准与选定候选的驻留峰值

后一条件是连续地址分配的关键，容量总和约束本身不能排除碎片。

记 $\chi_b = 1$ 表示缓冲区被输入复制节点使用，否则为零。一次换出的额外搬运量按题面为 $(2 - \chi_b)s_b$ 。因此目标为

$$D = \sum_{(b,j) \in \mathcal{M}} (2 - \chi_b)s_b, \quad (5)$$

其中 $\mathcal{M}$ 为所有换出事件的集合，重复换出必须重复计费。换入时间统一为 $2s_b + 150$ ；换出时间在 $\chi_b = 1$ 时为零，否则同为 $2s_b + 150$ 。

6.2 分配和牺牲对象选择

沿问题一候选序列维护各类缓存空闲区间。新申请先选择能容纳它的最小空闲区间，以降低剩余碎片。空间不足时，排除当前操作引用的缓冲区，从同类型驻留对象中选择牺牲者。第一种准则最大化距离下一次使用的序列距离 Δ_b ；第二种最大化 $\Delta_b / [(2 - \chi_b)s_b]$ ，以兼顾未来使用距离与搬运代价。无后续使用但尚不能逻辑释放的对象取序列尾部之后的距离，仍保留其配对换入与最终释放。

换出后立即释放该驻留段的地址，在后续使用前重新申请连续区间并插入换入节点。换入允许采用不同偏移。每次换出对应一对新增节点，原始节点总数为 N 时，第 j 次的编号为 $N + 2j$ 、 $N + 2j + 1$ ，其中 j 从零开始。

若受保护缓冲区本身分散在多个地址区间，使其他所需对象无法放入，即使总量不超过容量，单纯换出非保护对象也可能失败。此时将当前所需缓存类型的驻留对象显式换出，再按大小递减重装本次操作的全部对象。清空后的连续空间保证容量检查通过的操作能够重新装入；代价可能较大，故仅作为可行性恢复机制。禁止将此过程处理成无成本的内存整理。

算法每次空间申请扫描缓存单元并比较牺牲者，开销受缓存长度和换出次数共同控制。以 $C = \max_t C_t$ 、 M 为换出次数，其分配和牺牲者扫描可保守界为 $O((N + M)(C + |B|))$ ；未来使用位置通过队列预先索引。题目给定容量固定，但一般化分析中不能忽略 C 。

6.3 结果与物理验证

问题二允许调整问题一的顺序，因此在编号、缓存贪心和深度优先的最佳适配候选中，先选择搬运量最小者，再以时间打破平局。其结果不必与问题一最小峰值候选相同。

表 4: 缓存优先方案的额外搬运量与执行时间

计算图	搬运量	换出次数	周期
Conv_Case0	44766	56	472038
Conv_Case1	75546	2116	1210619
FlashAttention_Case0	3620	23	47509
FlashAttention_Case1	32852	212	225902
Matmul_Case0	34944	273	194331
Matmul_Case1	460800	3600	1800001

所有方案通过逐单元的地址占用检查，确保没有越界、重叠或非驻留使用。矩阵乘大图的搬运量仍然较大，说明有限缓存下的大量复用数据不能长期驻留。当前方法没有给出最小搬运量下界，因此不能将此数值解释为无法进一步降低的必要开销。

7. 搬运量约束下的流水优化

7.1 完整执行时间模型

在原图上加入换入换出依赖和地址复用依赖，得到驻留图 $G_R = (V_R, E_R)$ 。换出须等待本驻留段的全部使用者，换入须等待对应换出，后续使用者须等待换入。若地址被复用，则新驻留段的开始节点等待旧驻留段的释放节点。无换出时，该关系正好退化为题面的释放到申请边。

对给定序列，同一执行单元的前一条指令记为 $h(v)$ ，实际时间由

$$S_v = \max \left(0, \max_{u: (u,v) \in E_R} F_u, F_{h(v)} \right), \quad (6)$$

$$F_v = S_v + c_v, \quad T = \max_{v \in V_R} F_v \quad (7)$$

递推得到：不存在前项时相应项取零。管理节点没有独占执行单元。这个模型允许不同单元重叠，但绝不允许绕过缓存复用所需的同步。

7.2 地址轮转与固定驻留图重排

最佳适配往往重复使用低地址，即使其他区间暂时空闲，也会增加相邻任务间的物理依赖。地址轮转从上次分配末端继续寻找空闲区间，绕回后再搜索。该操作可能改变碎片和换出次数，因此只有满足 $D \leq D_2$ 的方案才保留。初始候选还包括估计最早开始时间和新增缓存量的加权排序，最终都由完整评价模型重新计算。

进一步固定已选方案的所有驻留段顺序及地址关系，仅重新安排执行单元上的指令顺序。首先按已有合法序列逆序计算

$$r_v = c_v + \max_{u: (v,u) \in E_R} r_u, \quad (8)$$

然后维护每个执行单元按 r_v 排序的就绪队列。管理节点就绪后优先处理；操作节点在各队列队首间按最早可开始时间择优，同分取较长下游路径。另比较直接按关键路径择优的规则。由于只改变拓扑序，不改驻留对象、偏移与换出事件，额外搬运量保持不变。重排后按换出实际出现顺序重新编号新增节点，并同步更新配对换入编号，保证文本附件次序一致。

固定驻留图也限制了可搜索空间：保留的物理复用顺序可能排除更优的地址方案。它是可行的局部优化方法，不是同时优化所有物理布局的精确算法。优先队列维护和下游路径计算开销为 $O((N + M) \log(N + M) + |E_R|)$ ，还需计入构造复用边时扫描缓存单元的开销。

7.3 优化效果

表 5: 问题三最终结果及相对问题二的时间降低

计算图	搬运量	周期	降低比例 (%)
Conv_Case0	44766	465892	1.30
Conv_Case1	75546	1044075	13.76
FlashAttention_Case0	3620	37744	20.55
FlashAttention_Case1	32852	150499	33.38
Matmul_Case0	34944	173274	10.84
Matmul_Case1	460800	1749773	2.79

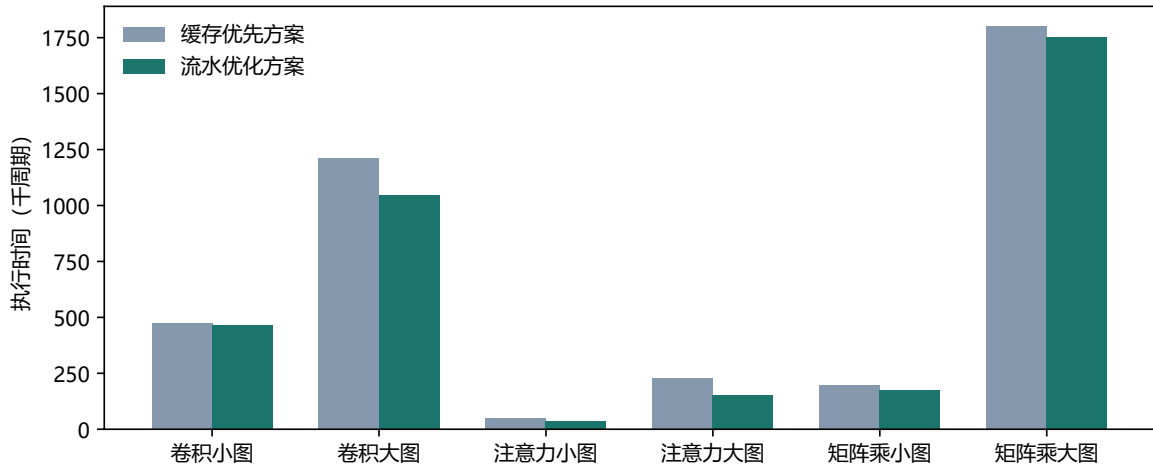


图 2: 缓存优先方案与流水优化方案的执行时间

六个图的搬运量均保持不变，时间改善存在明显差异。注意力大图改善最多，说明该图在既有地址约束下仍有较大的异构执行单元交错空间；卷积小图改善较少，表明仅调整指令顺序的收益有限。该解释基于当前候选表现，不推广为所有同类算子的固定规律。

图 3 将原始操作与换入换出分色。空白区间表示相应执行单元等待，竖向重叠表示不同单元同时运行。换出耗时为零的事件不形成可见条块，但仍保留在调度序列及依赖检查中，避免通过图形省略误认为没有换出。

8. 候选敏感性与验证

8.1 启发式敏感性

每图比较八种操作顺序、两种换出准则，并对距离换出增加地址轮换，共二十四个初始候选。固定驻留图优化从问题二方案和初步时间方案分别出发，再比较两种优先级。全部参数组合一致应用于六图，没有对某个算子单独调参。图 4 表明，局部内存收益并不充分代表全图代价。先处理能释放大缓冲区操作，可能同时启动更多尚不能结束的依赖分支，反而抬高后续驻留量。深度优先的前驱访问顺序也会改变共享数据的重用距离。故本研究的敏感性结论是规则选择依赖图结构，不能将某个参数称为普遍最优。六图均为同一题面提供的测试集，本轮没有额外的独立图集来验证泛化能力。

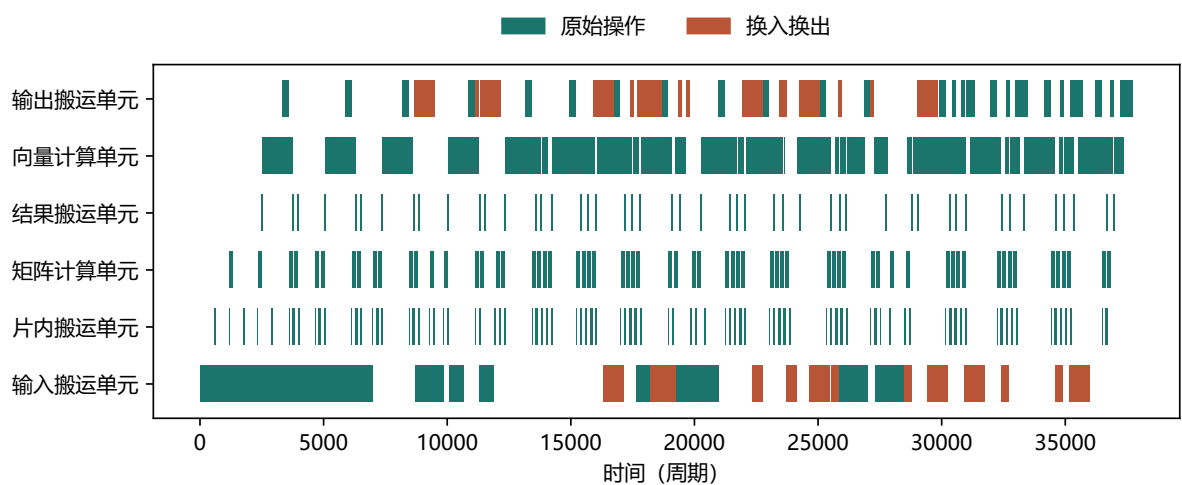


图 3: 注意力小图最终方案的完整执行单元流水

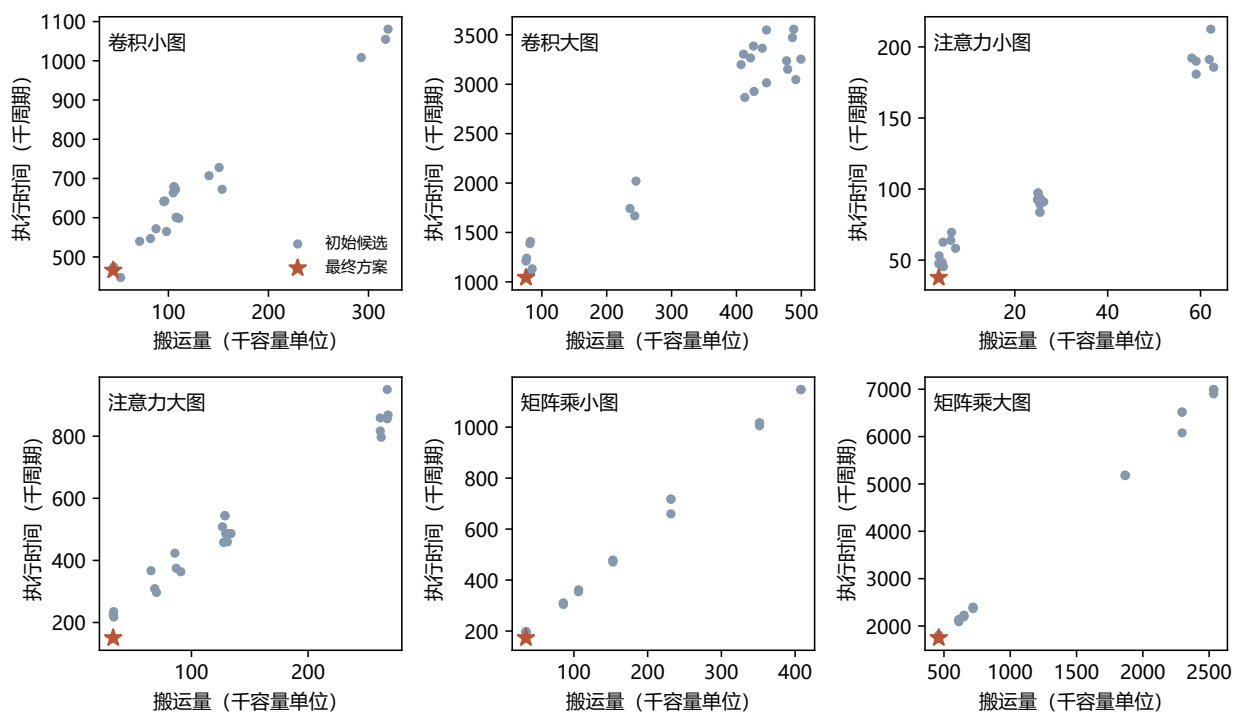


图 4: 初始候选的时间与搬运量分布及最终方案

8.2 手算例与反例校验

构造两个无依赖操作，分别运行于输入和输出搬运单元，耗时为 10 和 20 周期，缓冲区大小均为 4。使用互不重叠的地址时，总时间为 20；复用同一地址时，释放到申请依赖使总时间为 30。该例同时检验跨单元并行与物理地址串行化，避免评价器错误地把整个调度序列串行执行或完全忽略复用等待。

对输入复制缓冲区插入一次换出，额外搬运量为 4；在测试给定的顺序及复用关系下，总时间为 188 周期。对非输入复制缓冲区执行相应测试，搬运量为 8，总时间为 346。二者分别覆盖零耗时换出与双向搬运公式。人为注入地址重叠、拓扑逆序、偏移越界和节点重复，检查程序均拒绝这些错误附件。

最终六图均检查全部原始依赖和新增换出节点、连续空间占用、使用时驻留以及执行单元互斥。下界取各执行单元总工作量与原图关键路径的最大值；由于未求解全局优化，该下界只用于合理性检查，不能证明候选最优。校验程序与求解器共享输入解析，仍可能存在共同的题意理解偏差，需要结合题面复核。

9. 模型评价与结论

本方法把逻辑生命与物理驻留区分开来，能够显式计算换出释放地址产生的同步关系；通过先生成再独立读回验证，得到覆盖三个问题的完整文本结果。固定驻留图重排保持换出集合不变，因而能够在严格零搬运增量下改善流水时间。这是本轮六图中最稳定的优化来源。

方法的主要不足在于候选空间有限。最大驻留目标的搜索尚不充分，五图未超过收缩后的编号基准；贪心分配无法证明搬运量最小，碎片恢复可能产生较多额外换出；固定物理复用顺序又限制了时间优化空间。后续可在保留完整约束重放的基础上，引入有界局部交换和更精细的连续区间牺牲代价，并在新增图集上评价泛化，而不依据本轮六图直接作普遍性能承诺。

六个实例的最终时间改善为 1.30% 至 33.38%，搬运量均未增加。该结论针对题面抽象执行模型，不等价于真实芯片上的测量加速。特别是换出期间地址复用采用本文的驻留段解释，尚未获得官方评测程序的交叉确认。

本文模型推导、程序和实验分析使用 OpenAI Codex 辅助完成，工具精确模型标识及版本发布日期未核实。所有数值来自本地运行与文件重放，尚未经参赛队员独立复核。正式参赛稿应依据适用的人工智能使用规定，由队员理解、核验并用自身语言修订 [2]。

参考文献

- [1] 中国研究生数学建模竞赛组织委员会. 2025 年中国研究生数学建模竞赛 A 题：通用神经网络处理器下的核内调度问题及附件. 2025. 本研究使用用户提供的题面及六图原始数据.
- [2] 中国研究生数学建模竞赛组织委员会. “华为杯”第二十二届中国研究生数学建模竞赛人工智能工具及输出使用规定. 2025-08-21. <https://cpipc.acge.org.cn/sysFile/downloadFile.do?fileId=e3edba52b87c4bf7922f737c35c14b74>. 访问日期：2026-09-21.

A. 计算流程与附件说明

程序先对图作拓扑与数据检查，生成候选，再执行连续空间分配，最后重建驻留依赖并求各执行单元的完成时间。核心递推可写为以下伪代码，其中新节点的所有前驱均须已完成：

```
for node in topological_sequence:
    ready = max(finish[u] for u in predecessors[node])
    if node.has_execution_pipe:
        start = max(ready, pipe_finish[node.pipe])
    else:
        start = ready
    finish[node] = start + node.cycles
    if node.has_execution_pipe:
        pipe_finish[node.pipe] = finish[node]
makespan = max(finish.values())
```

无前驱时的最大值取零。前驱集合包含原始依赖、换入换出使用依赖和驻留段地址复用依赖；不先构造这些关系，以上递推不能给出有效评价。

附件按题面分为三个问题目录。问题一为每图一份调度序列；问题二和问题三另含缓冲区偏移与换出清单，共四十二份文本。每行一个调度节点，地址行采用缓冲区编号与偏移，换出行按实际换出次序给出缓冲区编号与新偏移。新增节点按该清单顺序成对编号。零换出方案须保留空换出清单，不能省略所需文件。

源程序及算法参数另行保留，所有运行均使用独立目录。本文未设置参赛队号，不以研究材料的文件名冒充正式上传命名；未执行官方评测程序，也未声称达到正式提交条件。